

M04 / STUDY COMPANION

M04 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	5
How to revise	5
DE04-01 Python setup and first program	5
60-second recall	5
Recall task	5
DE04-02 Variables, data types and operators	5
60-second recall	5
Recall task	5
DE04-03 Strings and formatting	6
60-second recall	6
Recall task	6
DE04-04 Conditions and boolean logic	6
60-second recall	6
Recall task	6
DE04-05 Lists, tuples, sets and dictionaries	6
60-second recall	6
Recall task	6
DE04-06 Loops and iteration	7
60-second recall	7
Recall task	7
DE04-07 Functions and scope	7

60-second recall	7
Recall task	7
DE04-08 Modules, imports and project structure	7
60-second recall	7
Recall task	7
DE04-09 Exceptions and debugging basics	8
60-second recall	8
Recall task	8
DE04-10 Files, paths, CSV and JSON I/O	8
60-second recall	8
Recall task	8
DE04-11 pandas foundations: load, inspect, filter and aggregate	8
60-second recall	8
Recall task	8
DE04-12 Practical Big-O intuition	9
60-second recall	9
Recall task	9
DE04-13 Coding drills: lists and strings	9
60-second recall	9
Recall task	9
DE04-14 Coding drills: dictionaries and sets	9
60-second recall	9
Recall task	9

DE04-15 Stack, queue, search and sort basics	10
60-second recall	10
Recall task	10
DE04-16 Integrated Python checkpoint and reproducible script habits	10
60-second recall	10
Recall task	10

Read this first

QUICK REVISION BOOK

Use this only after you have studied the learner book. Each lesson keeps the canonical language and core proof, plus one recall task. This is not a replacement for hands-on practice.

Coverage: 16 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to revise

Read the lesson card, close the page, explain it in your own words, then answer the recall task. If recall is weak, return to that lesson only - not the entire module.

DE04-01 | Python setup and first program

60-second recall

A Python source file is instructions; the interpreter executes them. Prove the interpreter with a version command, prove the file/path you ran, read the last exception line before editing, and preserve one failed attempt as debugging evidence.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

If you needed the Explained Review, close it before retrying. Create another file that prints a different label and calculates $9 * 6$. Introduce another syntax error deliberately, explain the final exception line before changing code, then repair it without reopening the review.

DE04-02 | Variables, data types and operators

60-second recall

Choose a Python type from business meaning, not appearance. IDs are often strings even when they contain digits. = assigns; == compares. Arithmetic produces values; comparisons produce booleans. Inspect `type(value)` before "fixing" a mysterious operation.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Use `shipment_id="S009"`, `cartons=7`, `unit_cost=425.5`, `delivered=False`. Calculate shipment cost and:

`needs_review = cost > 2500 and not delivered`

Predict both outputs before running. Then explain why not delivered is meaningful only because delivered is a boolean contract rather than a string such as "False".

DE04-03 | Strings and formatting

60-second recall

Strings are immutable. Preserve raw text, create cleaned/display values separately, and use `repr()` when invisible whitespace matters. An f-string formats a message; formatting is not identity proof. Never turn "looks similar" into "same entity" without a declared rule.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Given " Pen Blue ", "pen blue" (two internal spaces) and "PEN BLUE", normalize only surrounding spaces and case. Compare results. Do not silently collapse internal spaces. Explain what additional business rule would be needed before deciding those labels are equivalent.

DE04-04 | Conditions and boolean logic

60-second recall

Write the English rule first. `and` means all required conditions; `or` means at least one; parentheses make mixed rules auditable. Test boundaries and disqualifying cases. None can mean unknown and should not be silently treated as a business False unless the contract says so.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

A shipment is escalate only when `delayed_days >= 2` and `priority_customer` is True. Test 1, 2 and 3 delayed days for both customer flags, plus one `priority_customer=None` case. State what your code does with unknown priority and why.

DE04-05 | Lists, tuples, sets and dictionaries

60-second recall

List = ordered mutable sequence. Tuple = ordered fixed-style grouping. Set = unique membership. Dict = key -> value lookup. Choose by the requirement, not by whichever syntax you remember. Assignment can create two names for the same mutable object; copying behavior must be deliberate.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Represent warehouse zones and supervisors, a unique set of failed shipment IDs, a processing sequence, and a fixed (`country_code`, `year`) pair. Add a case where one zone legitimately has two supervisors and redesign the dictionary value rather than silently overwriting one name.

DE04-06 | Loops and iteration

60-second recall

A loop repeats a block for each item. Keep input and result state separate when possible. Use `continue` to skip a record deliberately, `break` only when early termination is truly the requirement, `enumerate` for position + value, and `.items()` for dict key/value pairs. Do not casually mutate the collection you are traversing.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Given shipment weights [12.5, -1, 0, 8.0, 15.2], create accepted and rejected lists, total only positive weights, and leave the source list intact. Explain why removing bad values from the same list during iteration is a weaker design.

DE04-07 | Functions and scope

60-second recall

Functions create named behavior boundaries. Parameters make dependencies visible; arguments are supplied values; `return` gives a result to the caller. Printing is observation, not a reusable return value. Avoid hidden mutable state and test more than one input.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Write `shipment_cost(shipment_id, cartons, cost_per_carton)` returning a dict with ID and cost. Test three cases including zero cartons. Then extend it to accept an explicit currency argument rather than reading a hidden global. Explain which inputs are measures and which are identifiers/labels.

DE04-08 | Modules, imports and project structure

60-second recall

A module is a Python file that can be imported. `Import` executes top-level module code, so keep side effects deliberate. Use a `main()` function plus `if name == "main":` for an explicit entry point. Avoid wildcard imports and local filenames that shadow standard/package modules.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Create `shipment_helpers.py` with `shipment_value(cartons, unit_cost)` and `normalize_hub(text)`. Create `run_shipment.py` with `main()`. Prove the helper can be imported without printing or starting the workflow. Explain which file owns reusable logic and which owns run orchestration.

DE04-09 | Exceptions and debugging basics

60-second recall

Preserve the traceback. Read the last exception line, inspect values/types at the failing expression, repair one assumption at a time and rerun. Catch only exceptions you understand and can handle safely. Parsing success does not prove business validity, and hiding an error is not recovery.

[Refresh source: Corey Schafer - Try/Except Blocks](#)

Recall task

Write `safe_positive_int(raw_value)` that either returns a positive integer or raises a clear `ValueError` for non-integer text, zero and negative values. Test "4", "0", "-2" and "four". Explain why parsing and business validation are two different checks.

DE04-10 | Files, paths, CSV and JSON I/O

60-second recall

Build paths from a known project/script base instead of a personal absolute path. CSV fields begin as text; parse deliberately. Validate required columns before transformation. Use UTF-8. JSON serialization needs JSON-compatible values. Read outputs back and compare critical values. Preserve raw inputs.

[Refresh source: Corey Schafer - File Objects](#)

Recall task

Create a fresh manifest for `M04_Retest_Shipments.csv`, read its CSV header with the standard `csv` module and validate these required fields: `shipment_id`, `hub`, `status`, `cartons`, `unit_cost`, `coordinator`. Add a deliberately wrong required field name and preserve the resulting missing-column error before repairing it.

DE04-11 | pandas foundations: load, inspect, filter and aggregate

60-second recall

Inspect shape, columns, dtypes, keys and missingness before transforming. Adding a column usually preserves grain; filtering changes population; `groupby` changes grain. State each output grain and reconcile grouped totals to a detail-level control. Top-N requires an explicit sort rule. Missing values are not automatically a label you should fill.

[Refresh source: Luke Barousse - Python for Data Analytics](#)

Recall task

Use `M04_Retest_Shipments.csv`. Calculate `shipment_value = cartons * unit_cost`; filter `Delivered`; aggregate `Delivered` value by `hub`; show top 3 `Delivered` shipments with a deterministic second key; count missing `coordinator` values over the population you explicitly name. Do not look at the review controls until your query/code is saved.

DE04-12 | Practical Big-O intuition

60-second recall

Big-O describes growth, not exact seconds. One pass is $O(n)$; nested full scans can be $O(n^2)$; set/dict membership is average $O(1)$; sorting is commonly $O(n \log n)$. Use the model to notice repeated work, then profile real programs when performance matters.

[Refresh source: freeCodeCamp - Data Structures and Algorithms in Python](#)

Recall task

Suppose each order must attach a region label from a customer mapping. Compare scanning every customer for each order with building one `customer_id` -> region dictionary first. State approximate build cost, lookup cost and total-growth intuition. Do not claim dictionary lookup is "always literally one CPU operation."

DE04-13 | Coding drills: lists and strings

60-second recall

State input/output/edge cases before code. Trace one example. Return values rather than only printing. Test blank and boundary inputs. A short, explainable solution with correct contract is stronger than a memorized clever one-liner.

Recall task

In a fresh file, implement:

```
normalize_words(text)
```

It should strip surrounding whitespace, split on runs of whitespace, lowercase tokens and return a list of tokens. Test blank text, one word and text with multiple spaces. This is deliberately different from `reverse_words`: the output type changes and no reversing is requested.

Continuous interview habit From this lesson onward, add one 15-20 minute closed-book Python micro-drill to study days.

DE04-14 | Coding drills: dictionaries and sets

60-second recall

Dicts map keys to values and are natural for counts/lookups. Sets are natural for membership/uniqueness. Combine set + list when you need fast "seen" checks and preserved first-occurrence order. Missing key and `key->None` are different. Composite tuple keys can represent multi-field identity.

Recall task

Write `duplicate_keys(records)` where each record has `order_id` and `line_no`. Return composite keys that appear more than once, without reporting the same duplicate key multiple times. Test no duplicates, one duplicate pair and two different duplicate pairs. Explain why checking only `order_id` would incorrectly flag legitimate multiple lines.

DE04-15 | Stack, queue, search and sort basics

60-second recall

Stack = LIFO. Queue = FIFO. Linear search works on arbitrary order and is $O(n)$. Binary search needs sorted input and is $O(\log n)$ lookup. Sorting commonly costs $O(n \log n)$. `sorted()` returns a new list; `.sort()` mutates. State preconditions and mutation behavior when explaining an algorithm.

[Refresh source: freeCodeCamp - Data Structures and Algorithms in Python](#)

Recall task

Use tasks ['ingest','validate','publish'] to contrast stack and queue removal order. Search target 12 inside [4,12,7,3] with linear search. Create a sorted copy before any binary-search attempt, and explain the extra sorting step. Add an absent-target test.

DE04-16 | Integrated Python checkpoint and reproducible script habits

60-second recall

A reproducible script has an explicit entry point, input contract, validation boundaries, deterministic output and documented run command. Validate before transformation, preserve raw input, reconcile important totals, prove a failure path and read outputs back. "It ran on my laptop" is not the same as reproducible engineering.

M04 move-on rule Do not move to M05 because you read all pages. Move when all lesson evidence is complete, any reviewed weakness has a successful changed retry, the four cluster checks are satisfactory, and the DE04-16 checkpoint has no unresolved critical defect. G03 is still later: M04 + M05 + M06 feed that protected Gate.

[Refresh source: Python Packaging User Guide](#)

Recall task

Only if the review/repair route assigns it, close all answer material and implement M04_Fresh_Checkpoint_TODO.py against M04_Retest_Shipments.csv. This changed scenario uses shipment/hub language, Delivered filtering and different controls.